If there are more threads than cores or processors, the operating system might also choose to schedule a thread on one core for one time slice and then on another core for the next time slice. This will therefore require transferring the cache lines for that thread's data from the cache for the first core to the cache for the second; the more cache lines that need transferring, the more time consuming this will be. Although operating systems typically avoid this when they can, it does happen and does impact performance when it happens.

Task-switching problems are particularly prevalent when lots of threads are *ready to run* as opposed to *waiting*. This is an issue we've already touched on: oversubscription.

8.2.5. Oversubscription and excessive task switching

In multithreaded systems, it's typical to have more threads than processors, unless you're running on *massively parallel* hardware. However, threads often spend time waiting for external I/O to complete or blocked on mutexes or waiting for condition variables and so forth, so this isn't a problem. Having the extra threads enables the application to perform useful work rather than having processors sitting idle while the threads wait.

This isn't always a good thing. If you have *too many* additional threads, there will be more threads *ready to run* than there are available processors, and the operating system will have to start task switching quite heavily in order to ensure they all get a fair time slice. As you saw in chapter 1, this can increase the overhead of the task switching as well as compound any cache problems resulting from lack of proximity. Oversubscription can arise when you have a task that repeatedly spawns new threads without limits, as the recursive quick sort from chapter 4 did, or where the natural number of threads when you separate by task type is more than the number of processors and the work is naturally CPU bound rather than I/O bound.

If you're simply spawning too many threads because of data division, you can limit the number of worker threads, as you saw in section 8.1.2. If the oversubscription is due to the natural division of work, there's not a lot you